PRIYANSHU SHARMA

Machine Learning | Computer Vision | RAG & Applied AI
New Delhi, India | princesharma99876@gmail.com | LinkedIn | GitHub

SUMMARY

B.Tech Computer Science (AI & ML) student who builds machine learning and applied AI systems end to end: data preparation, model training, evaluation, and live deployment. Built NORAG, a self-hostable Retrieval-Augmented Generation engine with hybrid retrieval, reranking and answer verification, and shipped three deployed ML applications (surface-crack segmentation, steel-defect classification and machine-failure prediction) using PyTorch, YOLO11, XGBoost, FastAPI, Docker and Render. Completed an ML internship at FlyRank AI (Jun to Sep 2026).

EXPERIENCE

Machine Learning Intern | FlyRank AI — Jun 2026 – Sep 2026

- Preprocessed and prepared datasets for model development within a collaborative ML team.
- Trained and evaluated machine learning models using structured evaluation practices, working from dataset to measured result.

PROJECTS

NORAG | Retrieval-Augmented Generation | Python, FastAPI, FastEmbed, Ollama, SQLite, React, TypeScript
Source code

- Built NORAG, a modular, self-hostable RAG engine from scratch that answers questions over uploaded PDF, Markdown, HTML and text documents with cited, verified answers, running 100% locally (FastEmbed embeddings, Ollama LLM, no API keys).
- Implemented hybrid retrieval combining dense vector search and BM25 through weighted Reciprocal Rank Fusion, an adaptive query router (semantic, exact, comparative, multi-hop) and cross-encoder reranking; added citation validation and claim verification against retrieved evidence.
- Built a retrieval evaluation module (Recall@K, Precision@K, MRR@K, nDCG@K) to compare dense, sparse, hybrid and hybrid+rerank systems, and served it all through a FastAPI REST API (API-key auth, rate limiting, health/readiness probes) with a React + TypeScript UI and 360+ test functions.

CrackVision | Surface crack segmentation | YOLO11n-seg, Ultralytics, OpenCV, Streamlit, Docker, Render
Live demo | Source code

- Trained a YOLO11n-seg instance-segmentation model to detect and outline surface cracks, reaching 63.8% mask mAP@50 (77.6% precision, 59.9% recall); compared 640 and 960 input resolutions and documented the precision/recall trade-off.
- Built a Streamlit inspection app with image upload, an adjustable confidence threshold, segmentation overlays, detection counts and local OpenCV camera inspection; Dockerized, deployed on Render, 20 pytest tests.

SteelGuard AI | Industrial computer vision | PyTorch, ResNet18, OpenCV, Streamlit, Docker, Render
Live demo | Source code

- Fine-tuned ResNet18 to classify six hot-rolled steel defect types on the NEU-DET dataset (1,800 grayscale images), reaching 98.33% validation accuracy; evaluated with a confusion matrix, per-class precision/recall/F1 and misclassification review.
- Added a "Steel Domain Gate", a second ResNet18 binary classifier trained against CIFAR-10 negatives, so off-domain uploads are rejected before any defect prediction; calibrated its decision threshold (0.9551) and documented the precision/recall trade-off.
- Routed predictions below a 70% confidence threshold to manual inspection, and shipped the full pipeline as a Dockerized Streamlit service on Render.

Predictive Maintenance System | XGBoost, FastAPI, Docker, Render
Live demo | Source code

- Built an end-to-end machine-failure predictor: EDA, preprocessing, feature engineering, XGBoost training and evaluation, with the model and preprocessing serialized for reuse.
- Served the model through a FastAPI REST API (auto-generated Swagger docs) and a browser interface that returns failure probability and a risk level.
- Containerized with Docker and deployed on Render; verified predictions were consistent across local, container and cloud environments.

Heat Exchanger Temperature Prediction | scikit-learn, XGBoost, pandas, NumPy
Source code

- Built a regression pipeline to recover true heat-exchanger temperature from noisy industrial sensor readings (Kaggle dataset).
- Benchmarked Linear Regression, Random Forest and XGBoost on MAE, MSE, RMSE and R^2; XGBoost performed best. Added residual, feature-importance and batch-prediction analysis.

TECHNICAL SKILLS

Languages: Python, SQL, TypeScript, C++ (learning)
Generative AI & RAG: Retrieval-Augmented Generation, hybrid search (dense + BM25), Reciprocal Rank Fusion, cross-encoder reranking, embeddings (FastEmbed), local LLMs (Ollama), query routing, answer and citation verification, retrieval evaluation (Recall@K, MRR, nDCG)
Machine Learning: PyTorch, torchvision, scikit-learn, XGBoost, transfer learning (ResNet18), CNNs, feature engineering, model evaluation, confidence calibration, YOLO11 (Ultralytics)
Computer Vision: OpenCV, image preprocessing, domain validation, instance segmentation, Pillow
Backend & Deployment: FastAPI, Pydantic, Uvicorn, REST APIs, SQLite, Streamlit, Docker, Render, API-key auth and rate limiting
Frontend: React, TypeScript, Vite, React Router
Scientific Computing: NumPy, pandas, Matplotlib, finite-difference methods (FTCS), numerical stability analysis
Tools: Git, GitHub, Linux, Jupyter, Google Colab, VS Code, pytest, uv, Ruff, ESLint, Prettier

EDUCATION

KCC Institute of Technology and Management — Expected 2029
B.Tech, Computer Science Engineering (Artificial Intelligence & Machine Learning)